Gustavo Franco Pereira

Mapa de Locais

Limeira – SP

2026

SUMÁRIO

1. INTRODUÇÃO 1

2. OBJETIVO 2

2.1. Objetivos Específicos 2

3. RESULTADOS ESPERADOS 3

4. LEVANTAMENTO DE REQUISITOS 4

4.1. Requisitos Funcionais 4

4.2. Requisitos Não Funcionais 5

4.3. Priorização 5

5. TECNOLOGIAS UTILIZADAS 7

6. ARQUITETURA DO PROJETO 8

6.1. Estrutura de Pastas 8

6.2. Responsabilidade das Camadas 8

6.3. Fluxo de Navegação 9

7. BANCO DE DADOS 10

7.1. Modelo da Coleção locais 10

7.2. Documentos Cadastrados 11

7.3. Regras de Segurança 11

8. OPERAÇÕES CRUD 13

8.1. Cadastro (Create) 13

8.2. Leitura (Read) 14

8.3. Edição (Update) 14

8.4. Exclusão (Delete) 15

9. TELAS DA APLICAÇÃO 17

9.1. Tela 1 – Mapa Principal 17

9.2. Tela 2 – Resultados 17

9.3. Tela 3 – Detalhes do Local 18

9.4. Recursos Complementares 19

10. TESTES 20

10.1. Tabela de Testes 20

10.2. Evidências 21

11. CONCLUSÃO 22

1. INTRODUÇÃO

Encontrar serviços e estabelecimentos em uma cidade desconhecida, ou mesmo na própria cidade, ainda é uma tarefa que depende de buscas espalhadas em diferentes fontes. Uma empresa deseja disponibilizar um aplicativo que reúna esses locais em um único lugar, permitindo que o usuário selecione a cidade, pesquise o que procura, filtre os resultados, abra os detalhes de um estabelecimento e marque os locais de seu interesse como favoritos.

Um problema comum em aplicações desse tipo é manter os dados fixos dentro do código: qualquer novo local exige uma nova versão do aplicativo, e as informações rapidamente ficam desatualizadas. Para resolver esse ponto, os locais precisam ser armazenados em um banco de dados na nuvem, de onde o aplicativo os busca, cadastra, altera e exclui em tempo real.

O projeto **Mapa de Locais** surge como resposta a esse desafio. Desenvolvido em Flutter e integrado ao Firebase Cloud Firestore, o aplicativo apresenta um mapa real com os estabelecimentos das cidades de São Paulo, Campinas e Jundiaí, uma tela de resultados com pesquisa e filtros e uma tela de detalhes com favoritos e horário de funcionamento. Novos locais são cadastrados diretamente pelo aplicativo, com a posição marcada tocando no próprio mapa, sem a necessidade de digitar coordenadas.

2. OBJETIVO

Desenvolver um aplicativo mobile em Flutter, integrado ao Firebase Cloud Firestore, que permita ao usuário localizar estabelecimentos em diferentes cidades por meio de um mapa interativo, pesquisar e filtrar resultados, consultar os detalhes de cada local e gerenciar os registros com operações completas de cadastro, edição e exclusão.

2.1. Objetivos Específicos

- Mapa interativo e seleção de cidade: exibir um mapa real com marcadores coloridos por categoria e permitir a troca entre São Paulo, Campinas e Jundiaí, com alternância entre visualização de ruas e satélite.
- Consulta em tempo real: ler os locais diretamente da coleção `locais` do Cloud Firestore, atualizando as telas automaticamente sempre que um documento for criado, alterado ou removido.
- Pesquisa e filtros: permitir a busca por nome ou categoria e a filtragem dos resultados por cidade e por categoria.
- Gerenciamento de locais (CRUD): cadastrar, editar e excluir locais por meio de formulários em *Bottom Sheet*, com a localização definida tocando no mapa e exclusão sempre precedida de confirmação.
- Detalhes e favoritos: apresentar todas as informações de um local, permitir marcá-lo como favorito e reunir os favoritos em uma aba própria.
- Padronização do horário de funcionamento: registrar o horário de cada dia da semana de forma estruturada, exibindo o horário do dia atual nos cartões e a semana completa na tela de detalhes.

3. RESULTADOS ESPERADOS

Espera-se que o Mapa de Locais ofereça ao usuário uma forma rápida e visual de encontrar estabelecimentos, substituindo listas estáticas por um mapa real em que cada local aparece com a cor e o ícone da sua categoria. A integração com o Cloud Firestore garante que todas as pessoas que utilizam o aplicativo vejam os mesmos dados, sempre atualizados, sem a necessidade de publicar uma nova versão a cada alteração.

Do ponto de vista de quem administra as informações, espera-se que o cadastro de novos locais seja simples e livre de erros: a posição é escolhida tocando no mapa, a cidade é sugerida automaticamente a partir do ponto marcado e o horário de funcionamento segue um padrão único para todos os registros. As mensagens de sucesso e de erro em cada operação e a confirmação antes de excluir tornam o uso seguro e previsível. Por fim, a organização do código em camadas (modelos, serviços, telas, componentes e diálogos) facilita a manutenção e a evolução do projeto.

4. LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos traduz a situação-problema e o wireframe proposto em funcionalidades concretas do aplicativo. Ele garante que o Mapa de Locais atenda às três telas obrigatórias (mapa principal, resultados e detalhes) e às operações de banco de dados exigidas, além de registrar as melhorias implementadas como extensão.

4.1. Requisitos Funcionais

Os Requisitos Funcionais descrevem o que o aplicativo deve fazer, ou seja, as ações disponíveis ao usuário em cada tela.

- **RF01 - Selecionar cidade:** o aplicativo deve permitir selecionar, no mínimo, as cidades de São Paulo, Campinas e Jundiaí, movendo o mapa para a cidade escolhida.
- **RF02 - Pesquisar:** o aplicativo deve permitir pesquisar locais por nome ou por categoria, tanto no mapa principal quanto na tela de resultados.
- **RF03 - Consultar Firestore:** os locais exibidos no mapa, nos resultados e nos favoritos devem ser lidos da coleção `locais` do Cloud Firestore, em tempo real.
- **RF04 - Filtrar:** a tela de resultados deve permitir filtrar os locais por categoria e por cidade.
- **RF05 - Cadastrar local:** o aplicativo deve criar um novo documento na coleção `locais`, com nome, categoria, cidade, horário, avaliação, distância, descrição e a posição marcada no mapa.
- **RF06 - Editar local:** o aplicativo deve alterar os dados de um documento existente, abrindo o mesmo formulário já preenchido.
- **RF07 - Excluir local:** o aplicativo deve excluir um documento somente após a confirmação do usuário.
- **RF08 - Detalhes:** o aplicativo deve exibir todas as informações do local selecionado, incluindo o horário de cada dia da semana.
- **RF09 - Favoritar:** o aplicativo deve alterar o campo `favorito` do documento e listar os locais favoritos em uma aba própria.
- **RF10 - Navegação:** o usuário deve conseguir navegar entre mapa, resultados e detalhes e retornar, além de alternar entre as abas Mapa, Favoritos e Sobre.
- **RF11 - Mapa real:** a tela principal deve apresentar um mapa real, com marcadores por categoria e alternância entre visualização de ruas e satélite.
- **RF12 - Marcar local no mapa:** a posição de um local deve ser definida tocando no mapa, sem a digitação manual de latitude e longitude.

- **RF13 - Horário padronizado:** o horário de funcionamento deve ser informado por dia da semana, com opção de marcar o dia como fechado e de copiar um horário para todos os dias.

4.2. Requisitos Não Funcionais

Os Requisitos Não Funcionais determinam como o aplicativo deve se comportar do ponto de vista técnico e de qualidade.

- **RNF01 - Tecnologia:** o aplicativo deve ser desenvolvido em Flutter/Dart.
- **RNF02 - Banco de dados:** os dados devem ser armazenados no Cloud Firestore, banco NoSQL da plataforma Firebase.
- **RNF03 - Configuração:** o projeto deve estar configurado com o FlutterFire CLI, utilizando o arquivo `firebase_options.dart` gerado automaticamente.
- **RNF04 - Plataforma:** a interface deve funcionar em dispositivos Android.
- **RNF05 - Organização:** o código deve ser organizado em pastas (models, services, screens, widgets e dialogs).
- **RNF06 - Estabilidade:** o aplicativo não deve apresentar erros durante a demonstração; a análise estática (`flutter analyze`) deve terminar sem problemas.
- **RNF07 - Segurança nas exclusões:** toda operação de exclusão deve pedir confirmação.
- **RNF08 - Feedback:** a interface deve apresentar mensagens de sucesso ou de erro em todas as operações de banco de dados.
- **RNF09 - Usabilidade:** a interface deve seguir as diretrizes do Material Design 3, com aparência próxima ao wireframe proposto.

4.3. Priorização

Para priorizar os requisitos foi utilizado o método MoSCoW, que separa o que é indispensável para a entrega daquilo que agrega valor ou fica para versões futuras.

- MUST HAVE (DEVE TER): funcionalidades sem as quais o aplicativo não atende à proposta da avaliação.
- SHOULD HAVE (DEVERIA TER): funcionalidades importantes, mas cuja ausência não impede o funcionamento básico.
- COULD HAVE (PODERIA TER): melhorias que enriquecem a experiência e foram implementadas como extensão.
- WON'T HAVE (NÃO TERÁ): itens discutidos, mas fora do escopo desta entrega.

Requisito	Priorização	Explicação
RF01, RF02, RF03, RF04	**Must have**	Seleção de cidade, pesquisa, leitura do Firestore e filtros formam o núcleo da proposta.
RF05, RF06, RF07	**Must have**	O CRUD completo é exigido pela avaliação e é o que diferencia o projeto de uma aplicação com dados fixos.
RF08, RF10	**Must have**	A tela de detalhes e a navegação entre as três telas são obrigatórias.
RNF01 a RNF08	**Must have**	Tecnologias obrigatórias, organização, estabilidade, confirmação de exclusão e mensagens de retorno.
RF09	**Should have**	Favoritos melhoram a experiência e fazem parte dos critérios, mas não bloqueiam o uso básico.
RF11, RF12, RF13	**Could have**	Mapa real, marcação da posição no mapa e horário padronizado foram implementados como extensão.
RNF09	**Could have**	Refinamentos visuais seguindo o Material Design 3 e o wireframe.
Autenticação, imagens (Storage), favoritos por usuário	**Won't have**	Desafios extras da avaliação que ficam para uma próxima versão do aplicativo.

Tabela 1 – Priorização dos requisitos pelo método MoSCoW. Fonte: autoria própria (2026).

5. TECNOLOGIAS UTILIZADAS

As tecnologias foram escolhidas seguindo o fluxo recomendado pela documentação oficial do Firebase para Flutter (Firebase CLI, FlutterFire CLI, `firebase_core` e `cloud_firestore`), acrescidas das bibliotecas necessárias para exibir um mapa real.

Tecnologia	Versão	Uso no projeto
Flutter / Dart	3.47.6 / 3.13.5	Interface e lógica da aplicação.
Firebase Core	firebase_core ^4.15.0	Inicialização do Firebase no aplicativo.
Cloud Firestore	cloud_firestore ^6.10.0	Banco de dados NoSQL onde ficam os documentos da coleção `locais`.
Firebase CLI e FlutterFire CLI	-	Configuração do projeto e geração do arquivo `firebase_options.dart`.
flutter_map	^7.0.2	Exibição do mapa interativo, das camadas de imagem e dos marcadores.
latlong2	^0.9.1	Representação de coordenadas e cálculo da cidade mais próxima do ponto marcado.
Esri World Street Map / World Imagery	-	Imagens do mapa de ruas e de satélite, sem necessidade de chave de API.
Material Design 3	-	Componentes visuais, tema e barra de navegação inferior.
Visual Studio Code e Android Emulator	-	Ambiente de desenvolvimento e testes.
Git e GitHub	-	Versionamento do código-fonte.

Tabela 2 – Tecnologias utilizadas. Fonte: autoria própria (2026).

6. ARQUITETURA DO PROJETO

O projeto segue a estrutura recomendada no tutorial da avaliação, separando as responsabilidades em camadas. As telas não acessam o banco diretamente: toda a comunicação com o Firestore passa pela classe `LocalService`, o que evita repetição de código e facilita a manutenção.

6.1. Estrutura de Pastas

```
lib/
├── main.dart                      Inicializa o Firebase e define o tema
├── firebase_options.dart          Gerado pelo FlutterFire CLI
├── models/
│   └── local.dart                 Classe Local (fromMap / toMap / copyWith)
├── services/
│   └── local_service.dart         Operações no Firestore e filtros
├── screens/
│   ├── main_screen.dart           Barra inferior: Mapa, Favoritos e Sobre
│   ├── mapa_screen.dart           Tela 1 – Mapa principal
│   ├── resultados_screen.dart     Tela 2 – Lista / resultados
│   ├── detalhes_screen.dart       Tela 3 – Detalhes do local
│   ├── favoritos_screen.dart      Aba de favoritos
│   └── perfil_screen.dart         Aba "Sobre" com informações do app
├── widgets/
│   ├── local_card.dart            Cartão de local usado nas listas
│   ├── local_form.dart            Formulário de cadastro e edição
│   ├── horario_picker.dart        Seletor de horário por dia da semana
│   ├── categoria_visual.dart      Ícone e cor de cada categoria
│   └── transicao_tela.dart        Animação de transição entre telas
└── dialogs/
    ├── confirm_delete.dart        Confirmação de exclusão
    └── selecionar_local_dialog.dart  Mapa em tela cheia para marcar o local
```

6.2. Responsabilidade das Camadas

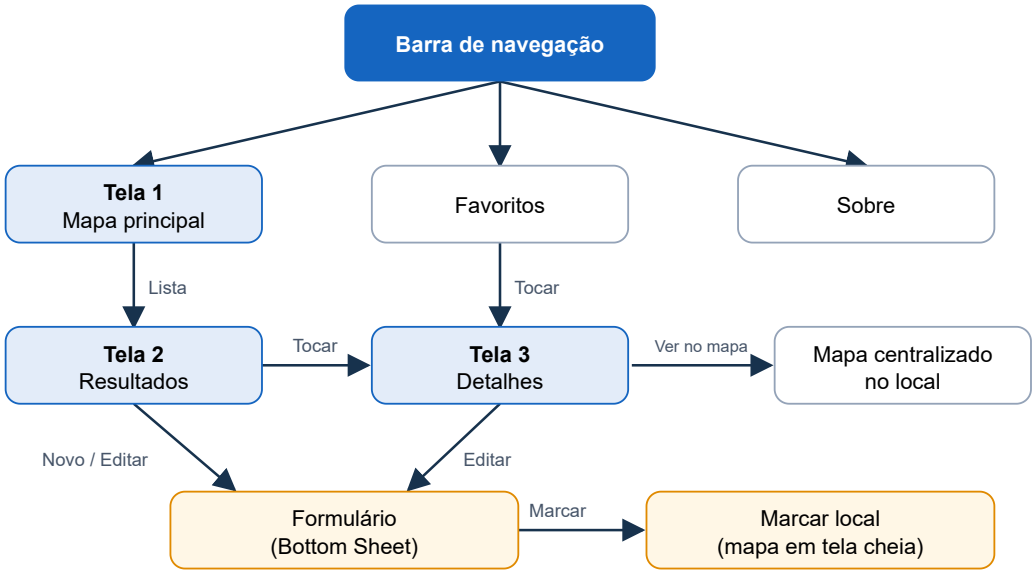
- **Models:** a classe `Local` representa um documento da coleção. O método `fromMap` converte os dados do Firestore em objeto e o `toMap` faz o caminho inverso; o campo `id` guarda o identificador do documento, usado nas operações de edição e exclusão.
- **Services:** a classe `LocalService` concentra as operações de banco (`listarLocais`, `adicionar`, `atualizar`, `excluir` e `alterarFavorito`) e a função `filtrarLocais`, que aplica a pesquisa e os filtros em memória.
- **Screens:** as telas montam a interface e reagem às ações do usuário, recebendo os dados em tempo real por meio de `StreamBuilder` ou de uma assinatura do *stream* do Firestore.
- **Widgets:** componentes reutilizáveis, como o cartão de local, o formulário e o seletor de horário, compartilhados entre várias telas.

- **Dialogs:** interações rápidas que não exigem uma nova tela principal, como a confirmação de exclusão e a escolha da posição no mapa.

6.3. Fluxo de Navegação

O aplicativo é aberto na tela de mapa, dentro de uma estrutura com barra de navegação inferior. A partir do mapa o usuário chega à lista de resultados e, dela, aos detalhes de um local; o botão "Ver no mapa" retorna ao mapa centralizado no local escolhido. O cadastro e a edição acontecem em um *Bottom Sheet*, sem criar uma quarta tela principal.

Figura 1 – Fluxo de navegação entre as telas



Fonte: autoria própria (2026).

7. BANCO DE DADOS

O banco de dados utilizado é o Cloud Firestore, banco NoSQL da plataforma Firebase, no projeto `mapa-locais-flutter-b6146`. Os dados são organizados em coleções e documentos; a aplicação utiliza uma única coleção, chamada `locais`, em que cada documento representa um estabelecimento. O identificador de cada documento é gerado automaticamente pelo Firestore no momento do cadastro.

7.1. Modelo da Coleção locais

Além dos campos sugeridos no tutorial, foram adicionados os campos `latitude` e `longitude`, que posicionam o marcador no mapa. O campo `horario` segue um padrão único, com o horário de cada dia da semana separado por barra vertical.

Campo	Tipo	Descrição
nome	String	Nome do estabelecimento.
categoria	String	Academia, Restaurante, Hospital, Museu, Parque, Escola, Hotel, Farmácia, Mercado ou Outro.
cidade	String	São Paulo, Campinas ou Jundiaí.
avaliacao	Number	Nota de 0 a 5.
distancia	Number	Distância em quilômetros.
horario	String	Horário por dia, no formato `Seg: 08:00-18:00 | ... | Dom: Fechado`.
descricao	String	Texto livre sobre o local.
favorito	Boolean	Indica se o local foi marcado como favorito.
latitude	Number	Latitude do ponto marcado no mapa.
longitude	Number	Longitude do ponto marcado no mapa.

Tabela 3 – Campos da coleção locais. Fonte: autoria própria (2026).

Exemplo de documento armazenado na coleção:

```
locais / dBG5tsTaz1Ol9l1GbLbP
{
  "nome": "Parque Ibirapuera",
  "categoria": "Parque",
  "cidade": "São Paulo",
  "avaliacao": 4.9,
  "distancia": 2.1,
  "horario": "Seg: 05:00-22:00 | Ter: 05:00-22:00 | ... | Dom: 05:00-22:00",
  "descricao": "Maior parque urbano de São Paulo, com pistas de corrida, lagos...",
  "favorito": true,
  "latitude": -23.5874,
  "longitude": -46.6576
}
```

7.2. Documentos Cadastrados

A coleção conta com doze locais cadastrados, quatro em cada cidade, atendendo ao mínimo de dez registros pedido no checklist de entrega. A tabela a seguir reproduz os documentos presentes no Cloud Firestore no momento da entrega.

ID do documento	nome	categoria	cidade	avaliacao	favorito
dBG5tsTaz1Ol9l1GbLbP	Parque Ibirapuera	Parque	São Paulo	4.9	true
pndUuEktmiHDAOSJPW05	MASP	Museu	São Paulo	4.8	false
b3xPw0RJMPri1lGEojl8	Academia Fitness Paulista	Academia	São Paulo	4.6	false
QJ7avWpgFfx2dHxdqcyO	Hospital Central SP	Hospital	São Paulo	4.5	false
Y50ehdQoVoq8XZFVTYua	Lagoa do Taquaral	Parque	Campinas	4.8	false
BzvsJL9kWDJBezjcsBub	Museu de Arte Contemporânea	Museu	Campinas	4.4	false
zVphZEt55B92lu35NJBw	Mercadão de Campinas	Mercado	Campinas	4.7	true
LYdcbi1UyWYc4XFmyjmD	Hospital de Clínicas Campinas	Hospital	Campinas	4.3	false
h8Of8f3UI9SpW546IoUe	Parque da Cidade	Parque	Jundiaí	4.7	false
5JQvPTLtaIQyZoNLRqqv	Academia Iron Gym	Academia	Jundiaí	4.5	false
YHYLLXEhb0i5vsbf8KYT	Cantina Bella Jundiaí	Restaurante	Jundiaí	4.6	false
NTAg2gC1K0yUjcPWry6E	Hospital São Vicente	Hospital	Jundiaí	4.4	false

Tabela 4 – Documentos da coleção locais no Cloud Firestore. Fonte: autoria própria (2026).

7.3. Regras de Segurança

Durante o desenvolvimento, o banco foi utilizado no modo de teste do Firestore, que facilita o aprendizado, mas permite leitura e escrita por qualquer pessoa que conheça o projeto. Essa configuração não é adequada para produção. Como próximo passo, o acesso de escrita deve ser restrito a usuários autenticados, mantendo a leitura pública e validando os campos obrigatórios, como no exemplo abaixo:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /locais/{id} {
      allow read: if true;
      allow create, update: if request.auth != null
        && request.resource.data.nome is string
        && request.resource.data.avaliacao >= 0
        && request.resource.data.avaliacao <= 5;
      allow delete: if request.auth != null;
    }
  }
}
```

8. OPERAÇÕES CRUD

As quatro operações básicas de banco de dados estão concentradas na classe `LocalService`. Como as telas utilizam `snapshots()`, qualquer alteração aparece automaticamente em todas elas, sem a necessidade de recarregar a lista.

Operação	Método no Firestore	Método lógico	Onde acontece no aplicativo
Create	`locais.add()`	POST	Botão "Novo local" na tela de resultados ou toque no mapa principal.
Read	`locais.snapshots()`	GET	Mapa, resultados e favoritos, em tempo real.
Update	`locais.doc(id).update()`	PUT / PATCH	"Editar" no cartão ou nos detalhes; coração de favorito.
Delete	`locais.doc(id).delete()`	DELETE	"Excluir" no cartão ou nos detalhes, após confirmação.

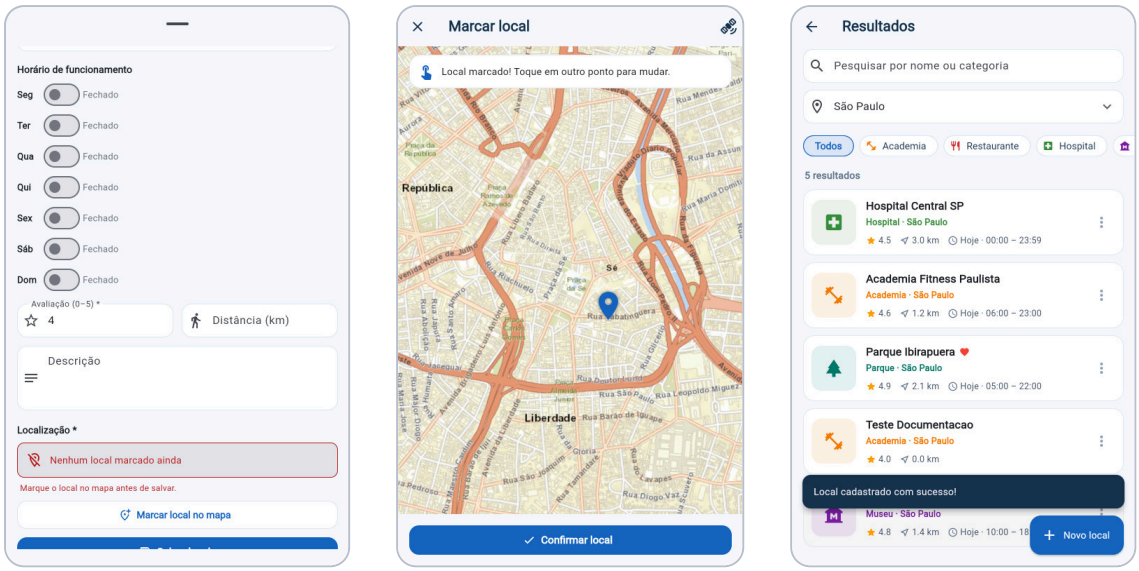
Tabela 5 – Operações CRUD do aplicativo. Fonte: autoria própria (2026).

8.1. Cadastro (Create)

O cadastro é feito em um formulário exibido em *Bottom Sheet*. O usuário informa nome, categoria, cidade, horário de funcionamento, avaliação, distância e descrição. Ao final do formulário, o botão "Marcar local no mapa" abre um mapa em tela cheia, centralizado na cidade escolhida, onde basta tocar no ponto desejado e confirmar. A cidade é ajustada automaticamente para a mais próxima do ponto marcado. Se o usuário tentar salvar sem marcar a posição, o formulário exibe uma mensagem de erro.

```
Future<void> adicionar(Local local) async {
  await locais.add(local.toMap());
}
```

Figura 2 – Cadastro de um novo local



a) Validação da posição b) Marcação no mapa c) Mensagem de sucesso

Fonte: autoria própria (2026).

8.2. Leitura (Read)

A leitura utiliza o método `snapshots()`, que devolve um *stream* atualizado a cada mudança na coleção. Na tela de resultados, um `StreamBuilder` trata os estados de carregamento, erro e lista vazia antes de exibir os cartões; no mapa, a tela assina o mesmo *stream* para redesenhar os marcadores.

```
Stream<List<Local>> listarLocais() {
  return locais.snapshots().map((snapshot) {
    return snapshot.docs.map((doc) {
      return Local.fromMap(doc.id, doc.data() as Map<String, dynamic>);
    }).toList();
  });
}
```

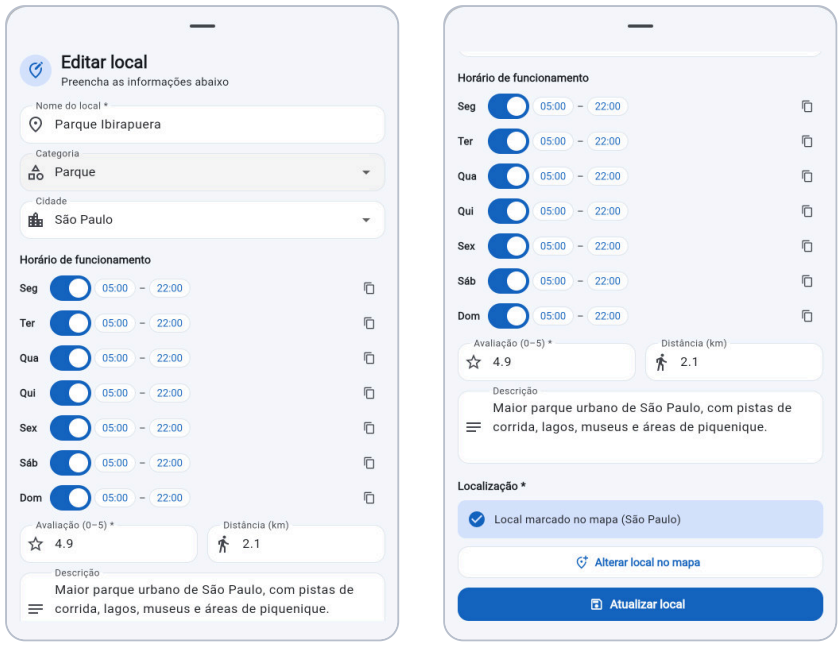
8.3. Edição (Update)

Ao selecionar "Editar", o mesmo formulário do cadastro é aberto já preenchido com os dados atuais, inclusive o horário de cada dia e a posição no mapa, que pode ser alterada pelo botão "Alterar local no mapa". O documento é identificado pelo seu `id`. O mesmo método de atualização é usado para o favorito, alterando apenas o campo `favorito`.

```
Future<void> atualizar(Local local) async {
  await locais.doc(local.id).update(local.toMap());
}

Future<void> alterarFavorito(String id, bool favorito) async {
  await locais.doc(id).update({'favorito': favorito});
}
```

Figura 3 – Formulário de edição



a) Dados e horário de funcionamento b) Localização e botão de salvar

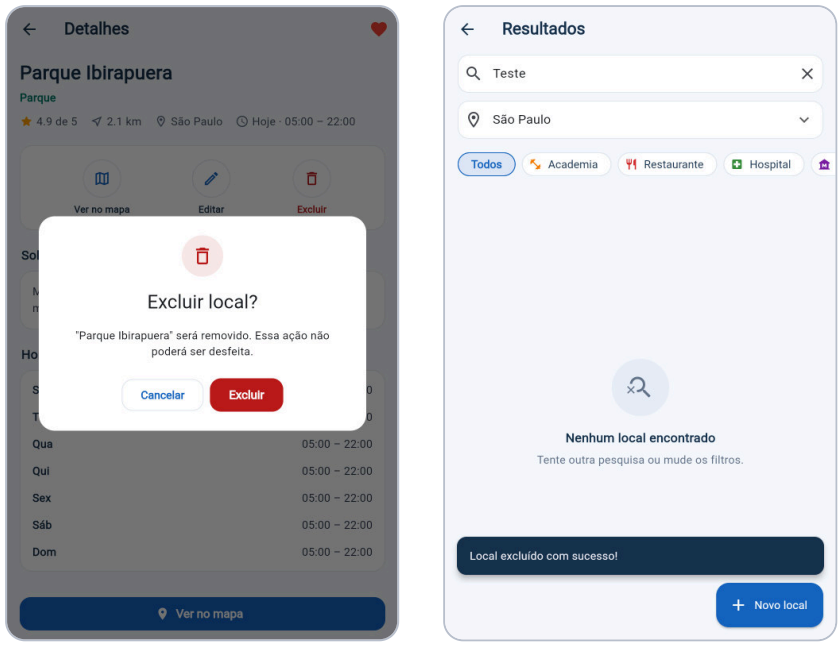
Fonte: autoria própria (2026).

8.4. Exclusão (Delete)

A exclusão nunca acontece diretamente ao tocar no botão. Antes, é exibido um diálogo de confirmação com o nome do local; somente se o usuário confirmar o documento é removido, e uma mensagem de sucesso é apresentada.

```
final confirmou = await showConfirmDeleteDialog(context, nome: local.nome);
if (confirmou == true) {
  await LocalService().excluir(local.id);
}
```

Figura 4 – Exclusão com confirmação



a) Diálogo de confirmação b) Local removido

Fonte: autoria própria (2026).

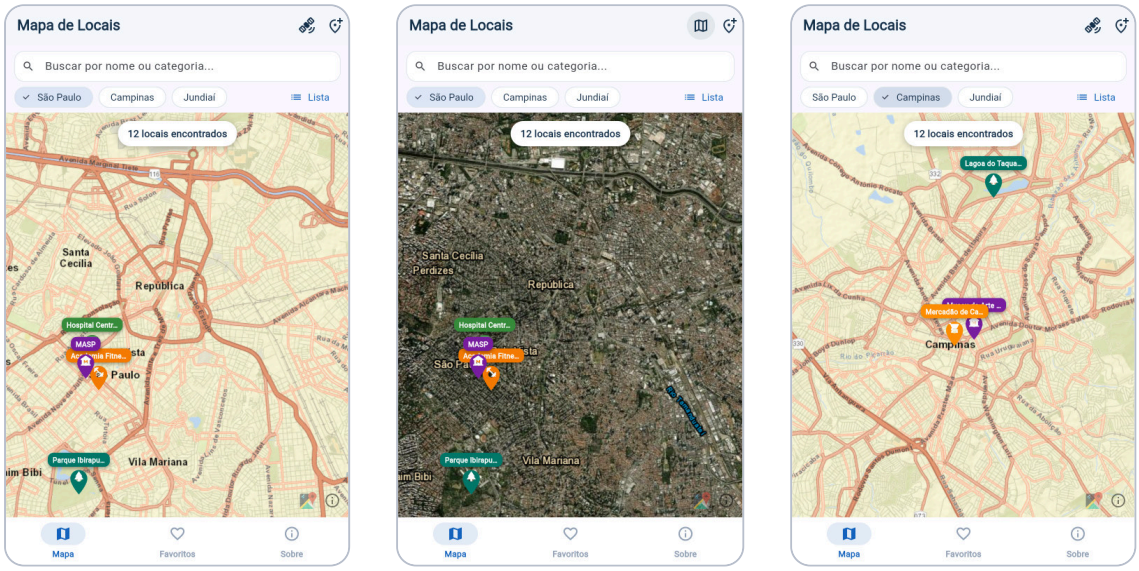
9. TELAS DA APLICAÇÃO

As três telas principais foram desenvolvidas com base no wireframe proposto. Os recursos adicionais, como o formulário e a marcação no mapa, aparecem em *Bottom Sheet* ou em telas sobrepostas, sem criar uma quarta tela principal.

9.1. Tela 1 – Mapa Principal

É a entrada da aplicação. No topo ficam o campo de pesquisa, a seleção de cidade (São Paulo, Campinas e Jundiaí) e o botão "Lista", que abre os resultados. O mapa real mostra os locais do Firestore com marcadores coloridos por categoria e um contador de locais encontrados. Na barra superior é possível alternar entre o mapa de ruas e o satélite e ativar o modo de cadastro tocando no mapa.

Figura 5 – Tela do mapa principal



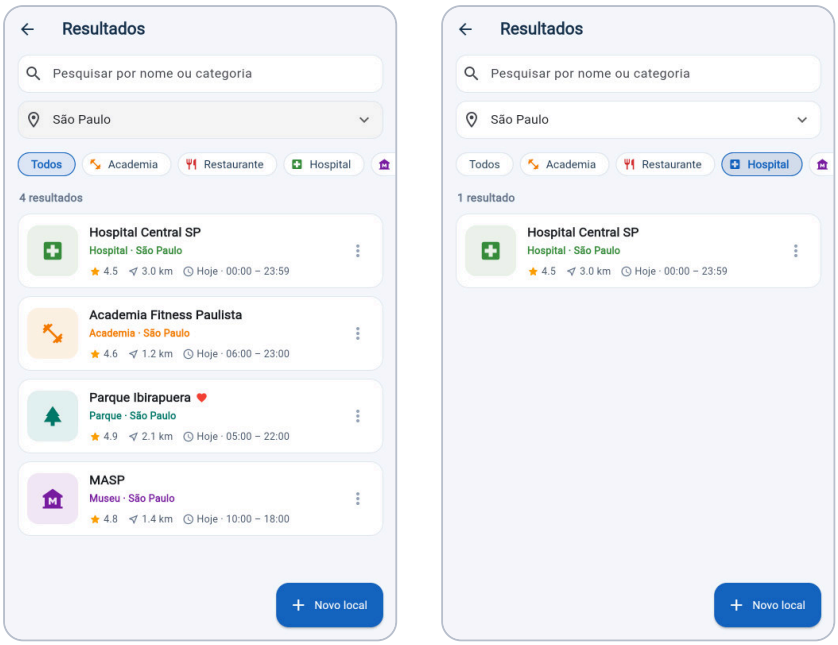
a) Mapa de ruas (São Paulo) b) Modo satélite c) Cidade de Campinas

Fonte: autoria própria (2026).

9.2. Tela 2 – Resultados

Apresenta os locais lidos do Firestore em cartões com nome, categoria, cidade, avaliação, distância e o horário do dia. Possui pesquisa, filtro por cidade, filtro por categoria em *chips*, contador de resultados e o botão "Novo local". O menu de cada cartão dá acesso às opções de editar e excluir, e o toque no cartão abre os detalhes.

Figura 6 – Tela de resultados



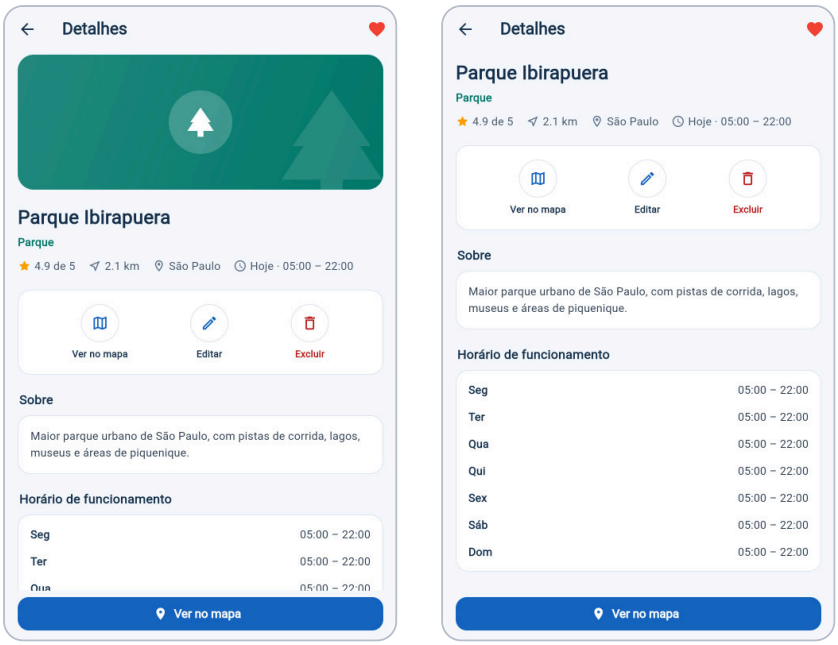
a) Locais de São Paulo b) Filtro por categoria

Fonte: autoria própria (2026).

9.3. Tela 3 – Detalhes do Local

Recebe o local selecionado e exibe todas as suas informações: destaque com a cor e o ícone da categoria, nome, categoria, avaliação, distância, cidade, horário do dia, ações rápidas (ver no mapa, editar e excluir), descrição e a tabela de horário da semana. O coração no topo altera o campo `favorito` no Firestore, e o botão "Ver no mapa" abre o mapa centralizado no local.

Figura 7 – Tela de detalhes



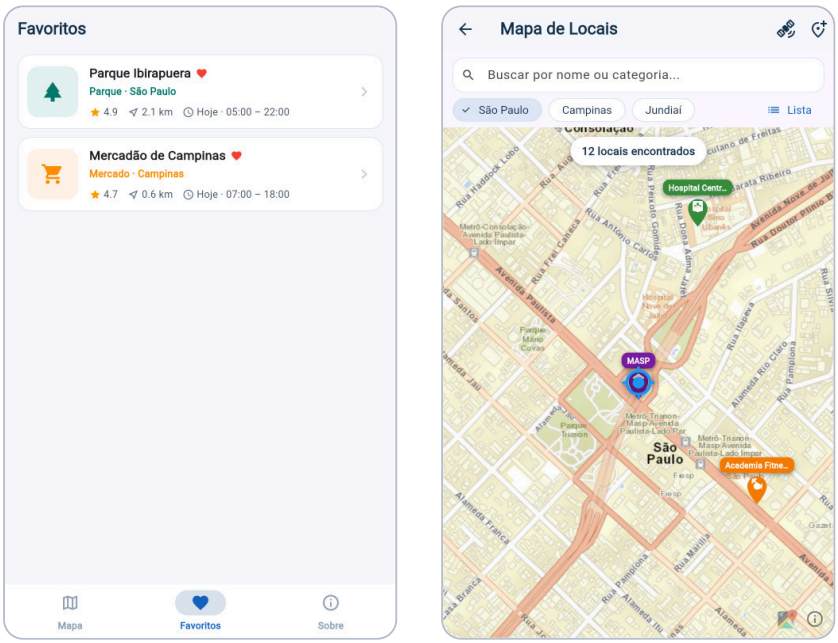
a) Informações e ações b) Horário de funcionamento

Fonte: autoria própria (2026).

9.4. Recursos Complementares

Como extensão, foi criada a aba Favoritos, que lista apenas os documentos com `favorito` igual a verdadeiro, e a ação "Ver no mapa", que leva o usuário de volta ao mapa com o local escolhido em destaque.

Figura 8 – Favoritos e "Ver no mapa"



a) Aba Favoritos b) Mapa centralizado no MASP

Fonte: autoria própria (2026).

10. TESTES

Os testes obrigatórios da avaliação foram executados com o aplicativo conectado ao Cloud Firestore. Nos testes de cadastro, edição, exclusão e favorito, além de observar a interface, o resultado foi conferido diretamente no banco de dados. O local criado para os testes T05 a T08 foi excluído no próprio teste T08, mantendo a coleção com os doze registros originais.

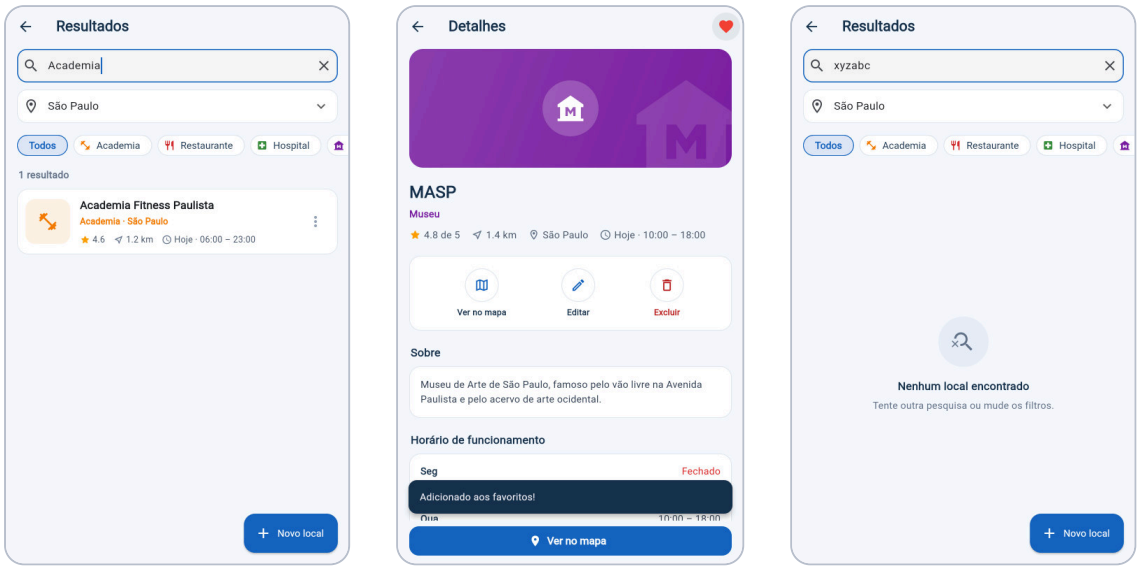
10.1. Tabela de Testes

ID	Teste	Procedimento	Resultado observado	Resultado
T01	Conexão	Abrir o aplicativo e verificar se os dados do Firestore aparecem.	Mapa exibe "12 locais encontrados" com os marcadores.	**Passou**
T02	Pesquisa	Pesquisar por "Academia".	Lista exibe "Academia Fitness Paulista" (1 resultado em São Paulo).	**Passou**
T03	Cidade	Selecionar Campinas.	Mapa se move para Campinas, exibindo os locais da cidade.	**Passou**
T04	Filtro	Selecionar uma categoria.	Filtro "Hospital" exibe apenas "Hospital Central SP".	**Passou**
T05	Create	Cadastrar um novo local e verificar no Firestore.	"Local cadastrado com sucesso!"; documento criado na coleção.	**Passou**
T06	Read	Fechar/reabrir a tela e verificar o registro.	Após recarregar o aplicativo, o local continua na lista.	**Passou**
T07	Update	Editar nome ou horário e confirmar a alteração.	Nome alterado no mesmo documento (mesmo ID) no Firestore.	**Passou**
T08	Delete	Excluir um local e confirmar que desapareceu.	Confirmação exibida; documento removido e lista atualizada.	**Passou**
T09	Favorito	Marcar e desmarcar um local.	Campo `favorito` passou para true e voltou para false.	**Passou**
T10	Navegação	Abrir resultados e detalhes e retornar.	Mapa → Resultados → Detalhes → Ver no mapa → retorno ao mapa.	**Passou**
T11	Erro	Simular ausência de dados/erro e verificar mensagem.	Pesquisa sem correspondência exibe "Nenhum local encontrado".	**Passou**

Tabela 6 – Resultado dos testes obrigatórios. Fonte: autoria própria (2026).

10.2. Evidências

Figura 9 – Evidências dos testes T02, T09 e T11



a) T02 – Pesquisa "Academia" b) T09 – Favorito c) T11 – Sem resultados

Fonte: autoria própria (2026).

Além dos testes funcionais, a análise estática do projeto com o comando `flutter analyze` foi executada ao final do desenvolvimento e terminou sem nenhum problema encontrado.

11. CONCLUSÃO

O projeto Mapa de Locais cumpre o objetivo de oferecer uma forma simples e visual de encontrar estabelecimentos em diferentes cidades. As três telas propostas no wireframe foram implementadas, e os dados deixaram de ser fixos no código para serem lidos, cadastrados, alterados e excluídos diretamente no Cloud Firestore, com atualização em tempo real em todas as telas.

Durante o desenvolvimento, a separação do código em camadas mostrou-se fundamental: concentrar o acesso ao banco na classe `LocalService` permitiu reutilizar as mesmas operações no mapa, nos resultados, nos detalhes e nos favoritos. Os maiores desafios foram a integração de um mapa real sem chave de API, a marcação da posição tocando no mapa no lugar da digitação de coordenadas e a padronização do horário de funcionamento, que passou a ser registrado dia a dia.

Como evolução, o aplicativo pode receber autenticação de usuários, regras de segurança restritas para escrita, imagens dos estabelecimentos no Firebase Storage, consultas diretamente no Firestore com `where()` e `orderBy()` e favoritos individuais por usuário, conforme os desafios extras propostos na avaliação.